MegaShop - Rangkuman Backend Firebase

Cheat sheet presentasi: file mana, search keyword apa, dan jawaban kalau ditanya backend/API.

1. Jawaban Inti Backend/API

- Backend aktif MegaShop adalah **Firebase full**, bukan folder `megashop_backend` FastAPI lama.
- Flutter berkomunikasi ke backend lewat Firebase SDK: Auth, Firestore, Storage, Messaging.
- API layer-nya bukan REST endpoint manual, tapi Firebase SDK calls seperti `FirebaseAuth`, `FirebaseFirestore`, `FirebaseStorage`, dan `FirebaseMessaging`.
- `await` = request sekali jalan ke backend; `StreamBuilder` + `.snapshots()` = data realtime dari Firestore.

> **Kalimat siap jawab:** Backend project ini memakai Firebase Backend-as-a-Service. Flutter tidak memanggil REST API manual, tetapi memakai Firebase SDK untuk authentication, database, file storage, realtime stream, dan notification.

2. File Backend Penting dan Search Keyword

Topik	File	Search keyword	Yang dijelaskan
Init Firebase	`lib/main.dart`	`Firebase.initializeApp`	Koneksi awal app ke Firebase sebelum `runApp`.
Session user	`lib/main.dart`	`currentUser`	Kalau user sudah login langsung ke `/home`.
Login email	`login_register_page.dart`	`signInWithEmailAndPassword`	Login ke Firebase Auth.
Register	`login_register_page.dart`	`createUserWithEmailAndPassword`	Buat akun di Firebase Auth.
Profile user	`login_register_page.dart`	`collection('users')`	Simpan uid, username, email, bio ke Firestore.
Google login	`login_register_page.dart`	`signInWithPopup` / `signInWithCredential`	Google provider untuk web/mobile.
Produk realtime	`home_page.dart` / `search_page.dart`	`collection('products')`	Ambil daftar produk realtime.
Story realtime	`home_page.dart`	`collection('stories')`	Ambil story realtime.
Upload story	`home_page.dart`	`FirebaseStorage` / `putData`	Upload gambar lalu simpan URL ke Firestore.
Create product/reel	`post_creation_page.dart`	`putData` / `putFile` / `products` / `reels`	Upload media dan buat document produk/reel.
Cart	`shared/state/cart_state.dart`	`carts` / `items` / `addItem`	Cart per user di Firestore.
Checkout order	`checkout_page.dart`	`collection('orders').add`	Buat order dari isi cart.
Chat	`conversation_page.dart`	`chats` / `messages`	Subcollection messages dan unread count.
Notification	`notification_service.dart`	`FirebaseMessaging` / `fcmTokens`	Token FCM dan listener chat/order.
Profile photo	`profile_page.dart`	`profile_photos` / `profileImageUrl`	Upload foto profil dan update referensi.

3. Database Firestore Schema

Collection	Isi utama	Dipakai untuk
`users`	uid, username, email, bio, profileImageUrl, fcmTokens	Auth profile, profile page, notification token.
`users/{uid}/addresses`	label, detail, createdAt	Alamat delivery di home dan checkout.
`products`	ownerId, name, price, description, imageUrl, createdAt	Home, search, product detail, seller/profile grid.
`reels`	ownerId, username, videoUrl, caption, price, likeCount	Reels page dan profile reels.

Collection	Isi utama	Dipakai untuk
`stories`	ownerId, username, imageUrl, createdAt	Story row dan story viewer.
`carts/{uid}/items`	productId, name, price, quantity, imageUrl, ownerId	Cart dan checkout.
`orders`	buyerId, sellerIds, participants, items, subtotal, tax, total, status	Checkout, notifications, order status.
`chats/{chatId}/messages`	senderId, text, createdAt	Realtime conversation.

4. Alur Backend per Fitur

Fitur	Alur backend singkat	Kode kunci
Login	Validasi form -> Firebase Auth -> route ke Home -> error ditampilkan lewat SnackBar.	`await FirebaseAuth.instance.signInWithEmailAndPassword(...)`
Register	Buat akun Auth -> buat dokumen `users/{uid}` di Firestore -> sign out -> user login manual.	`createUserWithEmailAndPassword + collection('users').doc(uid).set(...)`
Home produk	UI subscribe ke products order by createdAt. Kalau Firestore berubah, UI ikut update.	`collection('products').orderBy(...).snapshots()`
Upload produk	Pilih file -> upload ke Storage -> ambil downloadURL -> simpan metadata ke products/reels.	`await ref.putData(bytes); await ref.getDownloadURL();`
Cart	Cart disimpan per user. Product id dipakai sebagai document id. Kalau sudah ada, quantity naik.	`carts/{uid}/items/{productId}`
Checkout	Ambil cart -> hitung subtotal/tax/total -> add ke orders -> hapus cart item.	`collection('orders').add({...})`
Chat	Pesan disimpan di `chats/{chatId}/messages`. Parent chat menyimpan lastMessage dan unreadBy.	`chatRef.collection('messages').add({...})`
Notification	FCM token disimpan di users. App listen chats/orders untuk local notification.	`FirebaseMessaging.instance.getToken()`

5. Bedanya await dan StreamBuilder

- **await** dipakai untuk operasi satu kali: login, register, upload, add cart, checkout, delete item, update profile.
- **StreamBuilder** dipakai untuk realtime data: products, stories, cart list, chat messages, notifications.

```
await FirebaseFirestore.instance.collection('orders').add({...}); // write sekali jalan
FirebaseFirestore.instance.collection('products').snapshots(); // realtime stream
```

6. Error Handling yang Bisa Ditunjukkan

Bagian	Search keyword	Bentuk handling
Login	`FirebaseAuthException`	Switch `e.code`: user-not-found, wrong-password, invalid-email, too-many-requests.
Form auth	`_validateFields`	Validasi email/password sebelum request backend.
Upload post	`catch (e)` / Upload failed	Kalau Storage/Firestore gagal, tampil SnackBar.
Checkout	Your cart is empty / Order failed	Cek cart kosong dan catch error order.
Profile photo	Profile photo upload failed	Catch error upload dan feedback merah.
Notification	FCM token error / permission error	Try/catch saat permission dan token FCM.

7. Demo Cepat untuk Nilai Backend 10/10

- Register user baru, lalu tunjukkan document baru di collection `users`.
- Login, tutup app atau refresh, lalu tunjukkan session tetap masuk Home.
- Upload produk foto, lalu tunjukkan file masuk Storage dan document masuk `products`.
- Add to cart, lalu tunjukkan `carts/{uid}/items` bertambah.
- Checkout, lalu tunjukkan document baru di `orders` dan cart kosong.

- Kirim chat, lalu tunjukkan `chats/{chatId}/messages` update realtime.

8. Jawaban Q&A yang Aman

Pertanyaan dosen	Jawaban pendek
API-nya mana?	API layer-nya Firebase SDK. Flutter akses backend via FirebaseAuth, Firestore, Storage, Messaging, bukan REST endpoint manual.
Backend-nya apa?	Firebase Backend-as-a-Service: Auth untuk login, Firestore untuk database, Storage untuk file, Messaging untuk notifikasi.
Data exchange buktinya?	Ada `await` untuk write/read sekali jalan dan `StreamBuilder .snapshots()` untuk realtime Firestore.
Error handling ada?	Ada `FirebaseAuthException`, try/catch, form validation, loading state, empty state, dan SnackBar feedback.
FastAPI folder itu apa?	Itu backend lama/legacy. Versi project sekarang sudah migrasi ke Firebase full.
Realtime-nya dari mana?	Dari Firestore `.snapshots()`, contohnya products, stories, cart, dan chat messages.

9. Search Command Cepat di VS Code

`Firebase.initializeApp`	`FirebaseAuth.instance`	`FirebaseFirestore.instance`
`FirebaseStorage.instance`	`FirebaseMessaging`	`collection('orders').add`
`collection('chats')`	`.snapshots()`	`FirebaseAuthException`

Catatan: gunakan folder `megashop_flutter/lib` sebagai source utama. Folder `megashop_backend` adalah legacy FastAPI dan tidak perlu dijadikan backend aktif saat presentasi.